Take-Home: AI-Augmented Investment Pipeline

Context

You're the first engineering hire at a seed-stage VC firm. Partners spend ~10 hours/week scanning Product Hunt, YC, Hacker News, Twitter/X, and Crunchbase for promising startups, then writing memos by hand. Most candidates get passed on. Your job is to build the first version of an internal pipeline that automates the triage layer so partners can spend their time on the top 10%.

What to Build

A pipeline with three stages. Boundaries are deliberately loose — scoping is part of what we're evaluating.

1. Sourcing
Given a seed input (a topic query like *"AI agents for SMBs"*, a list of URLs, or a feed like the YC W25 batch), collect 10–20 candidate startups with: name, website, one-line description, founders/team signal where findable, and at least one freshness or traction signal (recent launch, funding, HN traction, GitHub activity — your pick).

Pick **one or two** sources and go deep.

2. Analysis
For each startup, produce a structured analysis covering:
- **Team** — founder backgrounds, prior exits, technical depth
- **Product** — what they actually do, in plain language
- **Market** — size hint, competitive landscape, why now
- **Risks / open questions** — what would kill this?
- **Score** (0–100) against your stated thesis

You define the thesis. Make it specific and defensible.

3. Recommendation
A one-page memo per startup ending in a clear call: **Pass / Watch / Take a meeting**, with rationale and the 2–3 things that would change your mind.

What We're Evaluating

Two things matter equally: **what you build** and **how you worked with AI to build it**.

We're hiring engineers who use AI well — to think, plan, prompt, debug, evaluate, and decide. A large share of your grade depends on whether we can actually see that working process by the time we finish reviewing your submission.

We are not going to tell you what to include to make it visible. Figuring that out is part of the exercise. If we finish reading your repo with no real picture of how you worked, we can't grade that dimension, and you'll lose those points. How you choose to leave a trail — and the quality of what you choose to show — is itself signal.

Beyond that, we look at: scoping and judgment, system design, code quality, and output quality.

Constraints

- **Stack:** your choice.
- **LLMs:** any model.
- **Data:** public sources only. Free tiers fine.
- **Scope:** if you're building a job queue, vector DB cluster, or React frontend — stop.

Submission

Send us a repo (or zip) plus a ~5 minute walkthrough video (Loom or similar) showing one startup end-to-end.

Everything else is your call. We need to be able to run the pipeline, read the outputs (commit them so we don't need to re-run), and form a picture of how it got built.

Rubric

Area	Weight	What we look for
Process & AI workflow visibility	40%	By the end of reviewing your submission, do we have a real picture of how you thought, planned, and worked with AI? The form is yours to choose; the *choice* of form is part of what's graded.
Output quality	20%	Memos a partner would actually skim. Scores defensible. Robust to bad or missing data.

Scoping & judgment	20%	Right problems picked. Right corners cut. Thesis is specific and held consistently.
System design	10%	Clean separation between stages. Replayable. No overengineering.
Code quality	10%	Readable, modular, appropriately tested.

Ground Rules

- Use AI freely. Coding assistants, agents, planning, prompt design — all fair game.
- Be honest about it. If an agent wrote a module end-to-end, say so. We don't penalize that; we penalize hiding it.
- Don't ghostwrite reflective writing. It's obvious.
- Don't pad. Volume isn't signal.

Anti-patterns

- Code dumped with no window into how it got built.
- Reflective writing that reads like a model wrote it about how a model was used.
- A trail clearly assembled after the fact to look thorough.
- A 12-source sourcing layer where each source returns 2 garbage results.
- Claims in memos with no traceable source.
- A thesis so broad ("good companies") that the score is meaningless.

What "Done" Looks Like

A partner can:
1. Run one command, point it at a topic, get memos out the other end.
2. Open any memo and understand the call in 60 seconds.

A reviewer can:
1. Spot-check one analysis and trust where its claims came from.
2. Walk away with a real picture of how you worked.

Ship that and stop.

Add `chiragmakkar` & hari@emsoft.com to private Github repository as collaborators.